Experiment Title: Implementation of Shortest Job First (SJF) Non-Preemptive CPU Scheduling Algorithm

Topic: Non-Preemptive CPU Scheduling with Arrival Time in C

1. Problem

In multi-programmed operating systems, CPU scheduling determines the order in which processes execute. The objective of this lab is to implement the **Shortest Job First (SJF) Non-Preemptive** CPU scheduling algorithm in C.

The program must:

- Accept the total number of processes (n), arrival time (AT), and burst time (BT) for each process.
- Schedule processes based on the shortest burst time among available processes.
- Handle different arrival times and break burst-time ties using first-come, first-served (FCFS) order.
- Compute Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for every process.
- Display the tabular summary, Total Turnaround Time, Total Waiting Time, Average TAT, and Average WT.

1.1 Environment & Tools

Item	Details	Item	Details
Operating System	Linux (Ubuntu 24.04 LTS)	**Host Machine**	`firefly`
User Account	`ratul`	**Language & Standard**	C (GNU C99)
Compiler	GCC (`gcc 13.x`)	**Execution Shell**	GNU Bash 5.2

2. Theoretical Solution

Shortest Job First (SJF) non-preemptive scheduling selects the process with the smallest execution (burst) time from the ready queue once the CPU becomes free.

2.1 Key Principles

- **Non-Preemptive Nature:** Once the CPU is allocated to a process, it cannot be interrupted until it finishes its entire burst time.
- **Ready Queue Eligibility:** A process i can only be chosen at time T if it has already arrived ($AT_i \leq T$).
- **Tie Breaking:** If two ready processes have the same burst time, the one with earlier arrival time is executed first.
- **CPU Idle State:** If no process is ready at the current time, the system advances time until a process arrives.

2.2 Formulas

For each process i, performance metrics are calculated using standard formulas:

- **Turnaround Time (TAT):** Total time taken from arrival to completion:

$$TAT_i = CT_i - AT_i$$

- **Waiting Time (WT):** Total time spent waiting in the ready queue:

$$WT_i = TAT_i - BT_i$$

- **Average Turnaround Time (Avg_TAT):** $\frac{\sum_{i=1}^{n} TAT_i}{n}$
- **Average Waiting Time (Avg_WT):** $\frac{\sum_{i=1}^{n} WT_i}{n}$

2.3 Gantt Chart Representation

Figure 1 illustrates the non-preemptive timeline for 4 processes (P_1: $AT = 0, BT = 7$; P_2: $AT = 2, BT = 4$; P_3: $AT = 4, BT = 1$; P_4: $AT = 5, BT = 4$):

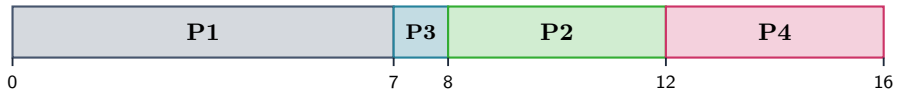


Figure 1: Gantt Chart of Non-Preemptive SJF Execution.

2.4 Algorithmic Approach

1. Read process count n, arrival times $arrival[i]$, and burst times $burst[i]$.
2. Sort processes in ascending order of arrival time using selection sort.
3. Initialize $current_time = 0$ and $completed = 0$.
4. Repeat while $completed < n$:
   - Scan all uncompleted processes where $arrival[i] \leq current_time$.
   - Select the one with the smallest $burst[i]$. If tied, select the smaller $arrival[i]$.
   - If a process is found: $CT = current_time + burst$, $TAT = CT - arrival$, $WT = TAT - burst$. Mark process complete, advance $current_time = CT$, increment $completed$.
   - If no process has arrived yet: advance $current_time$ by 1 unit.
5. Compute average turnaround time and average waiting time.

3. Expected Input/Output (I/O) Set

The input consists of integer counts and positive values for arrival and burst times. The output provides the detailed schedule metrics and overall averages.

3.1 Input Specification

- Total processes: $n = 4$
- Inputs per process: Process ID, Arrival Time (AT), Burst Time (BT).

3.2 Tabular Input and Expected Output

Process	Arrival Time (AT)	Burst Time (BT)	Completion Time (CT)	Turnaround Time (TAT)	Waiting Time (WT)
P1	0	7	7	7	0
P2	2	4	12	10	6
P3	4	1	8	4	3
P4	5	4	16	11	7

3.3 Expected Summary Metrics

- **Total Turnaround Time:** $7 + 10 + 4 + 11 = 32$
- **Total Waiting Time:** $0 + 6 + 3 + 7 = 16$
- **Average Turnaround Time:** $32/4 = 8.00$
- **Average Waiting Time:** $16/4 = 4.00$

4. Source Code

The complete C implementation of the Non-Preemptive SJF scheduling algorithm is given below:

```c
#include <stdio.h>

int main() {
    int n;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    int process[n], arrival[n], burst[n];
    int completion[n], turnaround[n], waiting[n];
    int is_completed[n];

    int i, j, temp, pos;
    int current_time = 0;
    int completed = 0;

    float totalTAT = 0;
    float totalWT = 0;
    float averageTAT, averageWT;

    // Input process details
    for (i = 0; i < n; i++) {
        process[i] = i + 1;

        printf("Enter arrival time of process %d: ", process[i]);
        scanf("%d", &arrival[i]);

        printf("Enter burst time of process %d: ", process[i]);
        scanf("%d", &burst[i]);

        completion[i] = -1;
        is_completed[i] = 0;
    }

    // Sorting processes based on arrival time
    for (i = 0; i < n; i++) {
        pos = i;

        for (j = i + 1; j < n; j++) {
            if (arrival[j] < arrival[pos]) {
                pos = j;
            }
        }

        // Swap arrival time
        temp = arrival[i];
        arrival[i] = arrival[pos];
        arrival[pos] = temp;

        // Swap burst time
        temp = burst[i];
        burst[i] = burst[pos];
        burst[pos] = temp;
```

```
54
55          // Swap process ID
56          temp = process[i];
57          process[i] = process[pos];
58          process[pos] = temp;
59      }
60
61      // Apply Shortest Job First (Non-Preemptive)
62      while (completed != n) {
63
64          int min_burst = 9999;
65          int index = -1;
66
67          // Find the shortest available process
68          for (i = 0; i < n; i++) {
69
70              if (arrival[i] <= current_time && is_completed[i] == 0) {
71
72                  if (burst[i] < min_burst) {
73                      min_burst = burst[i];
74                      index = i;
75                  }
76
77                  // If burst times are equal, select the one
78                  // that arrived first
79                  else if (burst[i] == min_burst &&
80                           arrival[i] < arrival[index]) {
81                      index = i;
82                  }
83              }
84          }
85
86          // If a process is available
87          if (index != -1) {
88
89              completion[index] = current_time + burst[index];
90              turnaround[index] = completion[index] - arrival[index];
91              waiting[index] = turnaround[index] - burst[index];
92
93              // Add to totals
94              totalTAT = totalTAT + turnaround[index];
95              totalWT = totalWT + waiting[index];
96
97              // Update current time
98              current_time = completion[index];
99
100             is_completed[index] = 1;
101             completed++;
102         }
103
104         // If no process has arrived yet
105         else {
106             current_time++;
107         }
108     }
109
110     // Calculate averages
111     averageTAT = totalTAT / n;
112     averageWT = totalWT / n;
113
114     // Printing the final process table
115     printf("\nProcess\tArrival Time\tBurst Time\tCompletion Time\tTurnaround Time\tWaiting Time\n");
116
117     for (i = 0; i < n; i++) {
118         printf("P%d\t%d\t\t%d\t\t%d\t\t%d\t\t%d\n",
119                process[i],
120                arrival[i],
121                burst[i],
122                completion[i],
123                turnaround[i],
124                waiting[i]);
125     }
126
127     // Print total and average values
128     printf("\nTotal Turnaround Time = %.0f", totalTAT);
129     printf("\nTotal Waiting Time    = %.0f", totalWT);
130
131     printf("\nAverage Turnaround Time = %.2f", averageTAT);
132     printf("\nAverage Waiting Time    = %.2f\n", averageWT);
133
134     return 0;
135 }
```

5. Execution of Code

The C program was compiled with GCC and executed in the Linux terminal environment.

5.1 Compilation and Terminal Output

ratul@firefly: ~/OS-Lab — bash

```
ratul@firefly:~/OS-Lab$ gcc sjf.c -o sjf
ratul@firefly:~/OS-Lab$ ./sjf
Enter the number of processes: 4
Enter arrival time of process 1: 0
Enter burst time of process 1: 7
Enter arrival time of process 2: 2
Enter burst time of process 2: 4
Enter arrival time of process 3: 4
Enter burst time of process 3: 1
Enter arrival time of process 4: 5
Enter burst time of process 4: 4
Process  Arrival Time  Burst Time  Completion Time  Turnaround Time  Waiting Time
P1       0            7           7                7                0
P2       2            4           12               10               6
P3       4            1           8                4                3
P4       5            4           16               11               7
Total Turnaround Time = 32
Total Waiting Time    = 16
Average Turnaround Time = 8.00
Average Waiting Time    = 4.00
ratul@firefly:~/OS-Lab$ _
```

5.2 Verification of Correctness

The execution results match theoretical calculations:

- **At $T = 0$:** P_1 is the only arrived process. It runs for 7 units until $T = 7$.
- **At $T = 7$:** Processes P_2 ($BT = 4$), P_3 ($BT = 1$), and P_4 ($BT = 4$) are available. P_3 has the shortest burst (1) and executes next until $T = 8$.
- **At $T = 8$:** P_2 and P_4 both have $BT = 4$. P_2 arrived earlier ($AT = 2 < 5$), so it gets the CPU and finishes at $T = 12$.
- **At $T = 12$:** P_4 runs and finishes at $T = 16$.
- All completion times, turnaround times, waiting times, and averages match the expected values exactly.

6. Conclusion

Key takeaways from this lab:

1. Implemented the Shortest Job First (SJF) Non-Preemptive scheduling algorithm in C.
2. Managed dynamic arrival times and implemented FCFS tie-breaking for equal burst times.
3. Verified accurate computation of Turnaround Time, Waiting Time, and their system averages against theoretical models.